Database Users, Roles, Privileges, and Access Control

Group 4(BrewPOS - Cafe Point-of-Sale System)

<table>
<tr><th>Section</th><th>Contents</th></tr>
<tr><td>Cover and links</td><td>GROUP 2: De Vera Miller P. , Pelia Jhon Francis, Macascas Mark Andrew, Lumbo Marianne, Butanas John David
Project Assigned: BREWPOS - Cafe Point-of-Sale System
Repository Link:
https://github.com/Nycri/brewpos/blob/main/docs/user-roles.md
Our Group Repository:
https://github.com/millerdevera6-glitch/park-it-database.git</td></tr>
<tr><td>Projects set up</td><td>The BrewPOS project was selected as the basis for implementing database access control. It is a Café Point-of-Sale system that uses PostgreSQL to manage menu information, staff records, and customer transactions.

The following tools and requirements were used for the setup:

<table><tr><th>Requirement</th><th>Description</th></tr><tr><td>Database Server</td><td>PostgreSQL</td></tr><tr><td>Database Management Tool</td><td>pgAdmin 4</td></tr><tr><td>Code Editor</td><td>Visual Studio Code</td></tr><tr><td>Version Control</td><td>Git and GitHub</td></tr><tr><td>Selected Project</td><td>BrewPOS</td></tr><tr><td>Database Name</td><td>grp_brewposdb</td></tr></table>
Setup Procedures:

1. Opened the BrewPOS GitHub repository.
2. Reviewed the project's README and database documentation.
3. Identified the database tables and existing user roles.
4. Prepared the local PostgreSQL environment.
5. Created a separate folder for the Activity 4 SQL scripts and</td></tr>
</table>

<table>
<tr><td></td><td>documentation.
6. Prepared the database access control implementation.</td></tr>
<tr><td>Access-control plan</td><td>The project implements Role-Based Access Control (RBAC) to assign database permissions according to user responsibilities.

Three database group roles were prepared:

<table><tr><th>Role</th><th>Responsibility</th><th>Access Level</th></tr><tr><td>Store Manager</td><td>Manages menu categories and products</td><td>Administrative</td></tr><tr><td>Cashier</td><td>Processes customer orders</td><td>Transactional</td></tr><tr><td>Viewer/Auditor</td><td>Reviews database records</td><td>Read-only</td></tr></table>
The permissions are based on the Principle of Least Privilege, which ensures that users receive only the access necessary for their assigned responsibilities.

The detailed access-control policies and permission matrix are documented in <code>access-control-plan.md</code>.</td></tr>
<tr><td>SQL scripts summary</td><td>Three SQL scripts were prepared to implement and verify database access control.</td></tr>
</table>

	SQL Script / **Purpose** `01_create_roles.sql` — Creates the PostgreSQL group roles for the Manager, Cashier, and Auditor. `02_grant_privileges.sql` — Assigns the appropriate database and table permissions to each role. `03_verify_permissions.sql` — Checks the created roles and verifies their assigned privileges. These scripts separate role creation, permission assignment, and verification to make the implementation easier to manage and review.
Verification results	The database access control implementation will be verified using PostgreSQL queries and permission tests. The following verification activities are included:

Verification Activity	Expected Result	Actual Result
Check created database roles	Three group roles should appear	[To be completed]
Check granted privileges	Permissions should match the access-control plan	[To be completed]
Manager updates a menu item	Allowed	[To be completed]
Cashier updates a menu item	Denied	[To be completed]

<table>
<tr><td></td><td>
<table>
<tr><td>Auditor inserts a new order</td><td>Denied</td><td>[To be completed]</td></tr>
<tr><td>Auditor views order records</td><td>Allowed</td><td>[To be completed]</td></tr>
</table>

Screenshots of SQL execution, role creation, permission verification, and access testing will be attached as supporting evidence.
</td></tr>
<tr><td>Issues and fixes</td><td>
The following table will document the problems encountered during the implementation.
<table>
<tr><th>Issue Encountered</th><th>Solution Applied</th><th>Status</th></tr>
<tr><td>PostgreSQL database connection problems</td><td>Checked the PostgreSQL server connection and database configuration.</td><td>[Update status]</td></tr>
<tr><td>Difficulty identifying the correct database tables</td><td>Reviewed the original project's database documentation.</td><td>[Update status]</td></tr>
<tr><td>Permission configuration errors</td><td>Reviewed the SQL GRANT statements and role assignments.</td><td>[Update status]</td></tr>
<tr><td>Verification of database privileges</td><td>Used PostgreSQL information schema queries to inspect permissions.</td><td>[Update status]</td></tr>
</table>
</td></tr>
<tr><td>Feedback to owners</td><td>The following suggestions are recommended to improve the database security and maintainability of BrewPOS:</td></tr>
</table>

	1. **Improved Role Management: Maintain clearly separated database roles to prevent users from receiving unnecessary privileges.** 2. **Stronger Database Security: Use restricted database accounts for application connections instead of administrative accounts.** 3. **Transaction Protection: Prevent unauthorized modification or deletion of completed customer transactions.** 4. **Access Monitoring: Maintain records of important database operations to support security monitoring and auditing.** 5. **Documentation Improvements: Provide clear documentation of database permissions and role assignments for future developers.**
AI usage declaration	**Artificial Intelligence tools, particularly ChatGPT, were used as assistance during the Activity 4 implementation.** **AI assistance was used for:** ● **Understanding PostgreSQL role-based access control.** ● **Preparing and reviewing SQL role creation and privilege assignment scripts.** ● **Organizing the access-control plan.** ● **Explaining database permission verification.** ● **Improving the structure and wording of the project documentation.** **The generated SQL scripts and documentation were reviewed and adjusted based on the actual BrewPOS database structure. Database execution and verification were performed separately by the project members.**

Group 5 PUP-SPOT (System for Property Ownership Tracker)

<table>
<tr><th>Section</th><th>Contents</th></tr>
<tr><td>Cover and links</td><td>GROUP 2: De Vera Miller P. , Pelia Jhon Francis, Macascas Mark Andrew, Lumbo Marianne, Butanas John David
Project Assigned: PUP-SPOT Campus Lost and Found System
Repository Link: https://github.com/grp05-dba/grp05_lostandfound
Our Group Repository:
https://github.com/millerdevera6-glitch/park-it-database.git</td></tr>
<tr><td>Projects set up</td><td>The assigned project was cloned from the GitHub repository of Group 05. The project files and SQL scripts were reviewed to understand the database structure and its required operations.
<table>
<tr><th>Project Information</th><th>Details</th></tr>
<tr><td>Project Name</td><td>PUP-SPOT Campus Lost and Found System</td></tr>
<tr><td>Repository</td><td>grp05_lostandfound</td></tr>
<tr><td>Database Name</td><td>grp05_lostandfounddb</td></tr>
<tr><td>Database Platform</td><td>PostgreSQL 18.6</td></tr>
<tr><td>Setup Tool</td><td>pgAdmin 4</td></tr>
</table></td></tr>
</table>

Access-control plan

The access-control plan was created to ensure that each database account only receives the permissions needed for its responsibilities. This follows the principle of least privilege.

Account / Role	Purpose	Assigned Permissions
PostgreSQL Administrator	Manages the database and user accounts	Full administrative access
lostandfound_app	Used by the application to perform database operations	Application-required SELECT, INSERT, UPDATE, and other necessary permissions
lostandfound_public	Used for public-facing application operations	Limited access based on the operations required by the system

The assigned permissions must match the actual SQL role-creation script. The public account should not automatically be considered read-only if it has INSERT or UPDATE privileges.

Security Measures:

- **Avoid using the PostgreSQL superuser for application connections.**
- **Give each account only the permissions it needs.**
- **Restrict unnecessary database and table access.**
- **Do not expose actual passwords in the repository.**
- **Verify both permitted and restricted operations.**

SQL scripts summary

The following SQL scripts were prepared and organized in the group's repository under the `activity4/lostandfound/` folder.

SQL Script	Purpose

<table>
<tr><td></td><td>
<table>
<tr><td>01_create_roles.sql</td><td>Creates the required database group roles and login accounts.</td></tr>
<tr><td>02_grant_privileges.sql</td><td>Assigns the appropriate permissions to each account.</td></tr>
<tr><td>03_verify_permissions.sql</td><td>Checks whether the assigned accounts have the expected privileges.</td></tr>
</table>
The scripts were executed using pgAdmin 4 and the appropriate PostgreSQL administrative account.

Execution Result: [Successful / With Issues]

Evidence: Insert screenshots showing the SQL scripts and successful execution results.
</td></tr>
<tr><td>Verification results</td><td>
Database permissions were tested to determine whether each account could perform its assigned operations.
<table>
<tr><th>Account Tested</th><th>SQL Operation</th><th>Expected Result</th><th>Actual Result</th></tr>
<tr><td>lostandfound_public</td><td>SELECT from items</td><td>Allowed</td><td>[SUCCESS]</td></tr>
<tr><td>lostandfound_public</td><td>UPDATE items</td><td>Denied, if not granted</td><td>[SUCCESS]</td></tr>
<tr><td>lostandfound_app</td><td>SELECT from items</td><td>Allowed, if granted</td><td>[SUCCESS]</td></tr>
<tr><td>lostandfound_app</td><td>INSERT into items</td><td>Allowed, if granted</td><td>[SUCCESS]</td></tr>
</table>
</td></tr>
<tr><td>Issues and fixes</td><td>
During the database setup and access-control implementation, the following situation occurred:

Issue Encountered: Default role inheritance and permission propagation required explicit schema-level usage grants (<code>USAGE</code> on schema <code>public</code>) before table-level privileges took effect for non-superuser accounts.

</td></tr>
</table>

	• **Solution / Action Taken: Added explicit `GRANT USAGE ON SCHEMA public TO ...` and `GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO ...` statements inside `02_grant_privileges.sql`.**
Feedback to owners	**After reviewing and setting up the assigned project, feedback was prepared for the owning group regarding the repository, database structure, and access-control requirements.** • **Project Owner: Group 05** • **Feedback:** ○ **The repository contains well-structured SQL scripts that can be used to set up the PostgreSQL database.** ○ **The database tables and foreign key relationships were reviewed and confirmed functional during setup.** ○ **The role and permission scripts were examined to align application access with security best practices.** ○ **Explicit schema usage permissions should be pre-factored into grant scripts to streamline onboarding for peer groups.** • **Feedback Status: Sent**
AI usage declaration	**AI tools were used as a supplementary resource during the completion of this activity. They assisted in explaining PostgreSQL role management, database privileges, SQL syntax, documentation organization, and troubleshooting.** **All SQL scripts were reviewed and tested by the group. The group remains responsible for checking the accuracy of the scripts, verifying database permissions, and ensuring that the submitted documentation reflects the actual implementation.** • **AI Tool Used: Gemini / ChatGPT** • **Purpose of AI Usage:** ○ **Understanding PostgreSQL users, roles, and privileges.** ○ **Assistance in organizing SQL scripts and documentation layout.** ○ **Guidance in troubleshooting database setup issues.** ○ **Assistance in preparing technical documentation.**

Group 8 (Campus Canteen Pre Order and Queue Management System)

Section	Contents
Cover and links	**GROUP 2: De Vera Miller P. , Pelia Jhon Francis, Macascas Mark Andrew, Lumbo Marianne, Butanas John David** **Project Assigned: (**Campus Canteen Pre Order and Queue Management System) **Repository Link:https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System.git** **Our Group Repository: https://github.com/millerdevera6-glitch/park-it-database.git**
Projects set up	**The selected project is the Campus Canteen Pre-Order and Queue Management System developed by BWIS IT Solutions. It is designed to allow students to pre-order food, manage orders, and monitor queue activities within a campus canteen.** **The repository was cloned from GitHub and examined to understand its database structure and requirements. PostgreSQL was used to implement the database access control.** **A new database named `canteendb` was created in PostgreSQL. The database access-control implementation was organized into three SQL scripts:** • `01_create_roles.sql` • `02_grant_privileges.sql` • `03_verify_permissions.sql` **These scripts were prepared to establish database roles, assign appropriate permissions, and verify the access-control configuration.**

<table>
<tr>
<td>Access-control plan</td>
<td>
he access-control plan was designed to control database access by assigning different roles and permissions. The implementation follows the principle of least privilege, which means that each database role receives only the permissions required for its intended responsibilities.

Database Roles:

<table>
<tr><th>Role</th><th>Type</th><th>Responsibility</th></tr>
<tr><td>canteen_app</td><td>LOGIN</td><td>Application account used by the backend system</td></tr>
<tr><td>canteen_app_grp</td><td>NOLOGIN</td><td>Group role for application database operations</td></tr>
<tr><td>canteen_readonly_grp</td><td>NOLOGIN</td><td>Group role that provides read-only database access</td></tr>
<tr><td>canteen_admin</td><td>LOGIN</td><td>Database administration and setup</td></tr>
</table>

Access Permissions:

<table>
<tr><th>Permission</th><th>Application Group</th><th>Read-only Group</th></tr>
<tr><td>SELECT</td><td>Allowed</td><td>Allowed</td></tr>
<tr><td>INSERT</td><td>Allowed</td><td>Not Allowed</td></tr>
<tr><td>UPDATE</td><td>Allowed</td><td>Not Allowed</td></tr>
<tr><td>DELETE</td><td>menu_item only</td><td>Not Allowed</td></tr>
<tr><td>Sequence USAGE</td><td>Allowed</td><td>Not Allowed</td></tr>
</table>

The application group is allowed to retrieve, insert, and update records. It is also given DELETE permission specifically for the <code>menu_item</code> table. Meanwhile, the read-only group can retrieve database records but cannot modify or delete them.
</td>
</tr>
<tr>
<td>SQL scripts summary</td>
<td>Three SQL scripts were prepared to implement and verify the database access-control configuration.</td>
</tr>
</table>

	File 1: 01_create_roles.sql **This script creates the PostgreSQL roles required by the system. It establishes the application group, read-only group, and application login account. The application login is assigned membership in the application group.** **File 2: 02_grant_privileges.sql** **This script assigns database and table permissions to the created group roles. It allows the application group to perform SELECT, INSERT, and UPDATE operations, with DELETE permission limited to the `menu_item` table. The read-only group receives SELECT permission on the database tables.** **File 3: 03_verify_permissions.sql** **This script checks whether the database roles and permissions have been properly configured. It uses PostgreSQL system information and permission-checking functions to examine role membership, table privileges, and specific access permissions.**
Verification results	**The database access-control implementation was checked using the three SQL scripts in PostgreSQL.** **The following screenshots serve as evidence of the implementation:** **Figure 1. Successful Creation of Database Roles** **[In Evidence]** **This screenshot shows the execution of the first SQL script, which creates the application group, read-only group, and application login role.** **Figure 2. Successful Assignment of Database Privileges** **[In Evidence]** **This screenshot shows the execution of the second SQL script, which assigns the required database and table permissions to the application and read-only groups.** **Figure 3. Verification of Database Roles and Permissions**

<table>
<tr><td></td><td>[In Evidence]

This screenshot shows the execution of the verification queries used to check the created roles, application role membership, and assigned table permissions.</td></tr>
<tr><td>Issues and fixes</td><td>Several issues were encountered during the database setup and implementation.

<table><tr><th>Issue Encountered</th><th>Solution Applied</th></tr><tr><td>Database name did not match the name specified in the SQL scripts.</td><td>Created a new database and standardized its name to <code>canteendb</code>.</td></tr><tr><td>PostgreSQL reported that some roles already existed.</td><td>Checked the existing roles and avoided recreating roles that were already present.</td></tr><tr><td>Database permissions could not be assigned when the required roles were unavailable.</td><td>Executed the role-creation script before the privilege-assignment script.</td></tr><tr><td>Database commands needed to be executed in the correct database connection.</td><td>Selected <code>canteendb</code> in pgAdmin before executing the database permission scripts.</td></tr></table>
These issues helped improve the understanding of PostgreSQL database connections, role creation, and permission management.</td></tr>
</table>

Feedback to owners	**The following recommendations are suggested to improve the database security of the Campus Canteen Pre-Order and Queue Management System:** **1. Implement role-based access control to ensure that each user has appropriate database permissions.** **2. Avoid using the database administrator account for regular application operations.** **3. Keep database passwords and sensitive credentials outside the public GitHub repository.** **4. Apply default privileges to ensure that newly created tables and sequences receive appropriate permissions.** **5. Regularly review database roles and privileges to prevent unauthorized access.** **6. Consider implementing additional access restrictions for student and vendor records through the application's authorization system.** **These improvements can help maintain database security and protect the information stored in the system.**
AI usage declaration	**Artificial Intelligence (AI) tools were used as learning and development assistance during the database access-control activity. AI assistance was used to understand the PostgreSQL role structure, prepare SQL scripts, troubleshoot database errors, and organize the technical documentation.** **The SQL scripts were reviewed and executed in PostgreSQL, while the implementation and verification process was performed using pgAdmin. AI was used as a supporting tool and not as a replacement for the actual implementation and testing.**

Group 6 (HabalGo - Ride and Booking Management System)

Section	Contents

<table>
<tr><td>Cover and links</td><td>GROUP 2: De Vera Miller P. , Pelia Jhon Francis, Macascas Mark Andrew, Lumbo Marianne, Butanas John David
Project Assigned: HabalGo - Ride and Booking Management System
Repository Link: https://github.com/jeanaroyo/HabalGo.git
Our Group Repository:
https://github.com/millerdevera6-glitch/park-it-database.git</td></tr>
<tr><td>Projects set up</td><td>The HabalGo repository was selected as the project for implementing database access control. It is a ride-booking management system that manages customer information, driver records, vehicles, locations, bookings, payments, and customer feedback.

The project was set up by cloning the GitHub repository and preparing its PostgreSQL database. The database was created, and the provided SQL schema was executed to establish the necessary tables.

The following database tables were prepared:

• customer
• driver
• vehicle
• driver_vehicle
• location
• booking
• payment
• rating_feedback</td></tr>
<tr><td>Access-control plan</td><td>The access-control plan was developed to organize database permissions according to the responsibilities of each role. Three database roles were established to apply the Principle of Least Privilege.

<table><tr><th>Database Role</th><th>Responsibilities</th><th>Assigned Permissions</th></tr><tr><td>HabalGo Admin</td><td>Manages and maintains database records.</td><td>SELECT, INSERT, UPDATE, DELETE</td></tr><tr><td>HabalGo</td><td>Performs normal application</td><td>SELECT, INSERT,</td></tr></table></td></tr>
</table>

<table>
<tr><td></td><td>
<table>
<tr><td>Application</td><td>database operations.</td><td>UPDATE</td></tr>
<tr><td>HabalGo Auditor</td><td>Reviews and monitors database records.</td><td>SELECT only</td></tr>
</table>
The administrator receives full access to the existing database tables, while the application role can retrieve, insert, and update records. The auditor is restricted to viewing records without modifying or deleting data.
</td></tr>
<tr><td>SQL scripts summary</td><td>
Three SQL scripts were created to implement and verify the database access-control system.
<table>
<tr><th>SQL File</th><th>Description</th></tr>
<tr><td>01_create_roles.sql</td><td>Creates three PostgreSQL roles: HabalGo Admin, HabalGo Application, and HabalGo Auditor.</td></tr>
<tr><td>02_grant_privileges.sql</td><td>Assigns database schema, table, and sequence permissions according to each role's responsibilities.</td></tr>
<tr><td>03_verify_permissions.sql</td><td>Checks the assigned permissions using PostgreSQL privilege-checking functions and information_schema views.</td></tr>
</table>
The scripts were executed in the PostgreSQL Query Tool using the HabalGo database.
</td></tr>
</table>

<table>
<tr>
<td>Verification results</td>
<td>
The database permissions were checked to determine whether each role received the correct access privileges.
<table>
<tr><th>Verification Test</th><th>Expected Result</th></tr>
<tr><td>Admin schema access</td><td>TRUE</td></tr>
<tr><td>Application schema access</td><td>TRUE</td></tr>
<tr><td>Auditor schema access</td><td>TRUE</td></tr>
<tr><td>Application INSERT permission</td><td>TRUE</td></tr>
<tr><td>Application DELETE permission</td><td>FALSE</td></tr>
<tr><td>Auditor SELECT permission</td><td>TRUE</td></tr>
<tr><td>Auditor UPDATE permission</td><td>FALSE</td></tr>
</table>
The verification queries demonstrated that each database role received permissions based on its assigned responsibilities. The application role was restricted from deleting records, while the auditor was limited to read-only access.
</td>
</tr>
<tr>
<td>Issues and fixes</td>
<td>
- Issue: Database was not yet created.
Fix: Created a new PostgreSQL database named
</td>
</tr>
</table>

	`grp00_habalgodb`. **- Issue: Database tables were missing.** **Fix: Executed the provided `schema.sql` file to create the required tables.** **- Issue: Database roles were not yet available.** **Fix: Executed `01_create_roles.sql` to create the required database roles.** **- Issue: Database permissions were not assigned.** **Fix: Executed `02_grant_privileges.sql` to assign the appropriate permissions.** **- Issue: Permissions needed verification.** **Fix: Executed `03_verify_permissions.sql` to check the assigned privileges.**
Feedback to owners	**The following recommendations are provided to improve the database security of HabalGo:** **1. Maintain separate database accounts for administrators and application users.** **2. Apply the Principle of Least Privilege to prevent unnecessary database access.** **3. Avoid using the PostgreSQL superuser account for regular application operations.** **4. Protect database credentials and avoid publishing passwords in the public GitHub repository.** **5. Regularly review database permissions to ensure that only authorized accounts have access.**
AI usage declaration	**Artificial Intelligence (AI) was used as an assistance tool during the activity to understand PostgreSQL access control, prepare SQL scripts, organize database permissions, and assist in writing the documentation.** **All generated scripts and instructions were reviewed and executed by**

	the student to understand the implementation and verify the database permissions.

Group 6 (MESA - Order and Reserve)

Section	Contents
Cover and links	**GROUP 2: De Vera Miller P. , Pelia Jhon Francis, Macascas Mark Andrew, Lumbo Marianne, Butanas John David** **Project Assigned:** MESA - Order and Reserve **Repository Link:** https://github.com/kathlenmaecallos11-arch/Restaurant-Online-Ordering-Table-Reservation-System **Our Group Repository:** **https://github.com/millerdevera6-glitch/park-it-database.git**
Projects set up	**The Restaurant Online Ordering and Table Reservation System was selected as the project for the access control implementation.** **The repository was downloaded from GitHub and prepared for database access control configuration. A PostgreSQL database named `mesa_db` was used for the implementation.** **Three SQL scripts were prepared to create database roles, assign privileges, and verify the access permissions.** **Files Created** **Filename** — **Description** **01_create_roles.sql** — **Creates the Admin, Staff, and Customer database roles.** **02_grant_privileges.sql** — **Assigns database permissions to each role.**

	03_verify_access.sql — Checks the created roles and assigned permissions.
Access-control plan	**The access control plan was created to organize database permissions according to the responsibilities of each user.** **User Roles** **Role — Description** **Admin — Has full access to manage database tables and records.** **Staff — Can view, insert, and update database records.** **Customer — Can view and insert permitted records.** **Access Permissions** **Operation — Admin — Staff — Customer** **View Records — Yes — Yes — Yes** **Insert Records — Yes — Yes — Yes**

	Update Records Yes Yes No **Delete Records** Yes No No **Manage Database Tables** Yes No No **Security Measures:** ● **Use separate database roles for different users.** ● **Assign permissions based on user responsibilities.** ● **Prevent unauthorized database operations.** ● **Protect database usernames and passwords.** ● **Regularly check assigned privileges.**
SQL scripts summary	## 01_create_roles.sql **This script creates three PostgreSQL login roles: `restaurant_admin`, `restaurant_staff`, and `restaurant_customer`. Each role has its own password and is intended for a different level of database access.** ## 02_grant_privileges.sql **This script grants database connection and schema permissions to the three roles. The Admin receives full privileges on existing tables and sequences, while Staff receives SELECT, INSERT, and UPDATE permissions. Customer receives SELECT and INSERT permissions.** ## 03_verify_access.sql **This script checks whether the three database roles were successfully created and displays their assigned table privileges using PostgreSQL system information views.**

<table>
<tr>
<td>Verification results</td>
<td>
The SQL scripts were prepared for execution in PostgreSQL using the database <code>mesa_db</code>.

The verification process includes:

<table>
<tr><th>Verification</th><th>Expected Result</th></tr>
<tr><td>Role Creation</td><td>Three database roles should appear in the PostgreSQL roles list.</td></tr>
<tr><td>Database Connection</td><td>All three roles should have CONNECT permission on mesa_db.</td></tr>
<tr><td>Schema Access</td><td>All three roles should have USAGE permission on the public schema.</td></tr>
<tr><td>Admin Privileges</td><td>Admin should have full privileges on existing tables and sequences.</td></tr>
<tr><td>Staff Privileges</td><td>Staff should have SELECT, INSERT, and UPDATE permissions.</td></tr>
<tr><td>Customer Privileges</td><td>Customer should have SELECT and INSERT permissions.</td></tr>
</table>
Note: Actual execution results should be confirmed using the PostgreSQL Query Tool before marking each verification as successful.
</td>
</tr>
<tr>
<td>Issues and fixes</td>
<td>- Issue: The database name did not match the original SQL script.</td>
</tr>
</table>

	• **Fix: Changed the database name to `mesa_db`.** **- Issue: Database roles were not yet created.** • **Fix: Created separate roles for Admin, Staff, and Customer.** **- Issue: Different users require different access permissions.** • **Fix: Assigned privileges based on each user's responsibilities.** **- Issue: Database permissions needed verification.** • **Fix: Created a verification SQL script to check roles and assigned privileges.** **- Issue: Unauthorized users may access restricted database operations.** • **Fix: Applied role-based access control to limit database operations according to assigned permissions.**
Feedback to owners	**The repository provides a useful example of a restaurant online ordering and table reservation system. Adding database role management and access control can help improve its security.** **Suggested Improvements:** • **Implement separate access levels for administrators, staff, and customers.** • **Restrict customer access to their own orders and reservations.** • **Protect sensitive customer information.** • **Apply stronger password management.** • **Regularly review database permissions.**

AI usage declaration	**AI assistance was used to help prepare the database access control plan, generate sample PostgreSQL role creation and privilege assignment scripts, explain SQL commands, and organize the project documentation.** **The generated SQL scripts were intended as implementation templates and should be tested and verified against the actual database structure before deployment.**

Group 3(On-the-Job Training (OJT) Record Management Application)

Section	Contents
Cover and links	**GROUP 2: De Vera Miller P. , Pelia Jhon Francis, Macascas Mark Andrew, Lumbo Marianne, Butanas John David** **Project Assigned: On-the-Job Training Record Management** **Repository Link: https://github.com/swvenz/Project-OJT-RMA-System.git** **Our Group Repository:** **https://github.com/millerdevera6-glitch/park-it-database.git**
Projects set up	**The OJT Record Management System repository was downloaded from GitHub and prepared for the implementation of database access control.** **PostgreSQL was used as the database management system. The database `ojt_managementdb` was used for the access control implementation.** **The project was prepared by identifying the database structure and creating SQL scripts to manage database roles and permissions.** **Setup Activities:** ● **Downloaded the project repository from GitHub.** ● **Prepared the PostgreSQL environment.** ● **Identified the project database.** ● **Created a separate folder for Activity 4 files.** ● **Prepared the SQL scripts for role creation, permission assignment, and verification.**

<table>
<tr>
<td>Access-control plan</td>
<td>
The project uses Role-Based Access Control (RBAC) to organize database permissions according to the responsibilities of each user.

The following roles were prepared:

<table>
<tr><th>User Role</th><th>Access Permissions</th></tr>
<tr><td>Administrator</td><td>Full database access, including viewing, inserting, updating, and deleting records.</td></tr>
<tr><td>OJT Coordinator</td><td>View and manage OJT records and student information.</td></tr>
<tr><td>Academic Supervisor</td><td>View student records and monitor OJT progress.</td></tr>
<tr><td>Company Supervisor</td><td>View assigned trainees and their OJT progress.</td></tr>
<tr><td>Student</td><td>View personal OJT records and submit required information.</td></tr>
</table>
Security Policies:

Apply the principle of least privilege.
Restrict unauthorized database access.
Assign permissions based on user responsibilities.
Limit database administration to authorized personnel.
Review database permissions to maintain security.

</td>
</tr>
<tr>
<td>SQL scripts summary</td>
<td>hree SQL scripts were prepared for the database access control</td>
</tr>
</table>

	implementation. **01_create_roles.sql** **This script creates five PostgreSQL group roles: `ojt_admin`, `ojt_coordinator`, `academic_supervisor`, `company_supervisor`, and `ojt_student`. These roles represent the different user responsibilities in the system.** **02_grant_privileges.sql** **This script assigns database connection and schema usage permissions to the created roles. It also grants the administrator full privileges over existing database tables and sequences.** **03_verify_access.sql** **This script checks the created database roles and displays the assigned table permissions. It helps verify whether the access control configuration was applied.**
Verification results	**The database access control implementation was prepared using PostgreSQL SQL scripts.** **The verification process includes:** ● **Checking whether the five database roles were successfully created.** ● **Checking database connection permissions.** ● **Checking schema access permissions.** ● **Checking the administrator's table and sequence privileges.** ● **Reviewing the assigned table permissions.** **The verification results are documented through screenshots of the SQL execution and query outputs.**
Issues and fixes	**- Issue: PostgreSQL was initially difficult to configure.** ● **Fix: Reviewed the PostgreSQL installation and database**

	connection settings. **- Issue: Database roles needed to be organized according to user responsibilities.** ● **Fix: Created separate group roles for the Administrator, OJT Coordinator, Academic Supervisor, Company Supervisor, and Student.** **- Issue: Incorrect database selection could cause SQL execution errors.** ● **Fix: Selected `ojt_managementdb` before executing the SQL scripts.** **- Issue: Database permissions needed to be checked after execution.** ● **Fix: Created a verification script to check the roles and assigned privileges.** **- Issue: Different users require different levels of database access.** ● **Fix: Prepared an RBAC plan to define permissions based on each user's responsibilities.**
Feedback to owners	**The OJT Record Management System provides useful functions for managing student training records and monitoring OJT activities.** **The following improvements are recommended:** ● **Implement more detailed database permissions for each user role.** ● **Ensure students can only access their own records.** ● **Restrict supervisors to viewing their assigned trainees.** ● **Protect sensitive student information from unauthorized access.** ● **Regularly review database accounts and permissions.** **These improvements can help strengthen the system's security and protect its stored information.**
AI usage declaration	**Artificial Intelligence (AI) was used as an assistance tool during the Activity 4 implementation. It helped explain PostgreSQL commands, prepare the access control plan, organize database roles, create SQL**

	scripts, and prepare the project documentation. **All suggested scripts and procedures should be reviewed and tested by the student before final submission. The implementation and verification results are based on the actual project environment.**

Group 4(BoardTrack: Boarding House & Rental Management System)

Section	Contents
Cover and links	**GROUP 2: De Vera Miller P. , Pelia Jhon Francis, Macascas Mark Andrew, Lumbo Marianne, Butanas John David** **Project Assigned: BoardTrack:Boarding House & Rental Management** **Repository Link: https://github.com/karlrobertc-art/BoardTrack/blob/main/docs/user-roles.md** **Our Group Repository: https://github.com/millerdevera6-glitch/park-it-database.git**
Projects set up	**The BoardTrack repository was cloned from GitHub and examined to understand its existing system structure and user roles.** **The project uses PostgreSQL as its database management system. The existing user roles documentation was reviewed to identify the different access levels and permissions required for the system.** **The PostgreSQL database used for the implementation was named `Boardtrack_db`.** **The following files were prepared for the access control implementation:** • `01_create_roles.sql` • `02_grant_privileges.sql` • `03_verify_permissions.sql` • `access-control-plan.md`
Access-control plan	**The access control plan was created to establish appropriate database**

permissions for the different users of BoardTrack.

The implementation follows the principle of least privilege, ensuring that users only receive the permissions necessary for their responsibilities.

User Role	Access Permissions
Administrator	Full access to create, read, update, and delete records.
Staff	Can create, read, and update records but cannot delete them.
Tenant	Read-only access to permitted records.
Application Runtime	Controlled database access for application operations.

PostgreSQL roles were prepared to provide an additional layer of security alongside the application's existing authorization system.

SQL scripts summary

File Name	Description
01_create_roles.sql	Creates the PostgreSQL roles required for the access control implementation.

	02_grant_privileges.sql — **Assigns database permissions to the administrator, application runtime, and read-only roles.** **03_verify_permissions.sql** — **Checks the created roles, role memberships, and assigned database privileges.** **Database: `boardtrack_db`**
Verification results	**The SQL scripts were prepared and executed through PostgreSQL Query Tool using the `boardtrack_db` database.** **The verification process included checking the existence of the PostgreSQL roles, their memberships, and their assigned privileges.** **The expected results were:** ● **Administrator role receives full table privileges.** ● **Application runtime role receives SELECT, INSERT, and UPDATE privileges.** ● **Read-only role receives SELECT privileges.** ● **Application runtime role does not receive DELETE privileges.** **Screenshots of the SQL execution and verification results serve as supporting evidence of the implementation.**
Issues and fixes	**ssue 1: Role Already Exists** ● **Problem: PostgreSQL reported that a role already existed when executing the role creation script.** ● **Cause: The role had already been created during a previous**

	execution. • Fix: Checked the existing PostgreSQL roles and avoided creating duplicate roles. **Issue 2: Role "boardtrack_db" Does Not Exist** • Problem: An error appeared while executing File 2: `ERROR: role "boardtrack_db" does not exist`. • Cause: The database name `Boardtrack_db` was mistakenly confused with a PostgreSQL role name. PostgreSQL treats database names and role names as separate objects. • Fix: Removed the unnecessary `\connect Boardtrack_db` command from File 2 when using pgAdmin Query Tool. The correct database was selected through the Query Tool connection before executing the script. The GRANT statements were retained with the correct role names: `boardtrack_admin_grp`, `app_runtime_grp`, and `app_readonly_grp`. **Issue 3: Permission Denied** • Problem: PostgreSQL may reject commands that create roles or assign database privileges. • Cause: The connected database account does not have sufficient administrative permissions. • Fix: Used an authorized PostgreSQL administrator account to execute the SQL scripts. **Issue 4: Tenant Access Restriction** • Problem: Read-only database privileges alone do not prevent tenants from viewing other tenants' records. • Cause: SELECT privileges allow access to all granted tables unless additional restrictions are implemented. • Fix: Recommended tenant-specific filtering through application authorization or PostgreSQL Row-Level Security (RLS). •
Feedback to owners	**The BoardTrack project already provides documentation for its application roles and access permissions. Implementing PostgreSQL role-based access control provides an additional security layer.** **The following improvements are recommended:**

	● **Maintain separate application and database permissions.** ● **Apply the principle of least privilege.** ● **Implement tenant-specific database restrictions.** ● **Store database credentials securely outside the public repository.** ● **Regularly review database privileges when new features are added.**
AI usage declaration	AI assistance was used to help understand the existing user role documentation, prepare the access control plan, organize PostgreSQL SQL scripts, troubleshoot database errors, and prepare the project documentation. The generated SQL scripts and recommendations were reviewed and tested as part of the Activity 4 implementation process.